

MASS-APP-015-V01 — Plugin Foundation, Manifest & Capability Registration

Document Control

Field | Value

Document ID | MASS-APP-015-V01

Version | 1.0

Status | Production Baseline v1.0

Authority | EWO-MASS-APP-015-V01

Date | 2026-08-03

1. Purpose

V01 establishes the governed vocabulary and registry through which MASS recognizes installable plugins and their declared capabilities without modifying platform core. A plugin is a signed, versioned package of declarations and artifacts. A capability is one callable or consumable contract exposed by that plugin.

2. Permanent Architecture

V01 owns plugin identity, manifest schema, publisher references, immutable published versions, capability declarations, dependency declarations, compatibility metadata, entry-point references, validation findings, registry records, discovery metadata, and installation-eligibility findings. It does not install, activate, invoke, execute, sell, or grant permissions.

Definitions:

- Plugin: governed package and lifecycle identity.
- Capability: declared unit with inputs, outputs, errors, side effects, permissions, and approval requirements.
- Pack: curated set of plugin references; not a plugin merge.
- Integration: external-system connectivity owned by APP-020/ENG-016 and referenced by a capability.
- Automation: governed orchestration consuming ENG-006/ENG-025; not hidden plugin execution.
- Agent: AI-assisted participant orchestrated through ENG-009 and declared as a capability type.
- Template: reusable artifact definition owned by APP-013 when applicable.

3. Role Mapping

Role | Baseline Role | Authority

Registry Viewer | Viewer | Discover eligible plugins and capabilities

Publisher Contributor | Contributor | Submit manifests and documentation

Registry Steward | Steward | Validate declarations and provenance

Capability Owner | Contributor extension | Maintain declared capability metadata

Registry Approver | Administrator | Publish or reject validated versions

Registry Administrator | Administrator | Configure classifications and eligibility policy

Publishers cannot approve their own manifests. Registration does not grant installation or execution.

4. Platform Consumption Map

Source | Consumption | Boundary

Constitution / ENG-001–027 | Authority and platform contracts | Remain authoritative

MASS-PLAN-001 | Product identifiers and roadmap | Roadmap-owned

APP-013 | Template and artifact references | APP-013-owned

APP-014 | Intelligence/advisory contracts | APP-014-owned

APP-021/022 when available | Administration/security contracts | No assumed ownership before availability

ENG-003/004/005/011/015/027 | Identity, authorization, events, audit, APIs, lineage | Platform-owned

Publishes `plugin.manifest.submitted`, `plugin.version.validated`, `plugin.version.published`, `capability.registered`, `plugin.deprecated`.

5. Gateway Inventory

Gateway | Purpose

ConstitutionGateway / EngineeringLibraryGateway | Validate declared authority and dependencies

ProductRoadmapGateway | Validate product references

DesignStudioGateway | Resolve APP-013 artifact contracts

IntelligenceContractGateway | Resolve APP-014 advisory contracts

AdministrationGateway / SecurityGovernanceGateway | Future APP-021/022 contracts when available

IdentityGateway / AuthorizationGateway | Principal and permission checks

EventGateway / AuditGateway | Events and immutable audit evidence

ApiContractGateway / LineageGateway | Interface and provenance validation

6. Manifest Contract

The canonical JSON Schema accompanies this volume. A manifest includes canonical name, display name, plugin classification, publisher, version, compatibility range, capabilities, dependencies, entry points, permissions, tenant eligibility, documentation, support, signing provenance, deprecation, and replacement.

Canonical names use lowercase reverse-domain or approved MASS namespace tokens and are immutable after first publication. Versions use semantic versioning. Published manifest content is immutable; correction creates a new version.

7. Capability Contract

Each capability declares one type: tool, workflow, agent, template, knowledge, dashboard, form, event, or integration. It records purpose, inputs, outputs, errors, side effects, permissions, approval mode, entry point, timeout, idempotency expectations, consumed contracts, emitted events, and documentation.

Side effects are classified none, internal_write, external_write, communication, financial, security, or operational. Approval is none, user_confirmation, steward, administrator, or executive; platform policy may require stricter approval but never weaker

approval.

8. Validation and Discovery

Lifecycle: Draft → Submitted → Structural Validation → Contract Validation → Security/Provenance Review → Approved → Published → Deprecated → Retired.

Validation checks schema, naming, uniqueness, signatures, publisher provenance, dependencies, compatibility, permissions, side effects, entry points, documentation, and replacement references. Findings remain immutable after decision.

Discovery returns only published, tenant-eligible, compatible capabilities visible to the principal. Discovery does not imply installation, permission, or invocation authority.

9. Data Model

Entity | Purpose | Immutable Condition

PluginPublisher | Publisher identity/provenance | Verified evidence immutable

Plugin | Stable canonical identity | Canonical name immutable

PluginVersion | Versioned manifest | After publication/rejection

CapabilityDeclaration | Declared capability contract | After publication

CapabilityCategory | Governed type/category | After activation

PluginDependency | Required/optional/peer/conflict reference | After publication

PluginEntryPoint | Invocation contract reference | After publication

ManifestSignature | Signing/provenance evidence | Always

ValidationFinding | Validation result | After review

InstallationEligibility | Tenant eligibility advisory | After decision

PluginAudit | Append-only audit | Always

10. API Contracts

Method | Path | Purpose | Role

POST | /plugins | Create plugin identity | Publisher Contributor

POST | /plugins/{id}/versions | Submit manifest version | Publisher Contributor

POST | /plugin-versions/{id}/validate | Run validation | Registry Steward

POST | /plugin-versions/{id}/publish | Publish version | Registry Approver

POST | /plugin-versions/{id}/reject | Reject version | Registry Approver

GET | /plugins | Discover eligible plugins | Registry Viewer

GET | /plugins/{id} | Read plugin/version history | Registry Viewer

GET | /capabilities | Discover capabilities | Registry Viewer

GET | /capabilities/{id} | Read capability contract | Registry Viewer

POST | /plugin-versions/{id}/eligibility | Assess tenant eligibility | Registry Steward

POST | /plugin-versions/{id}/deprecate | Deprecate version | Registry Approver

11. Security and Integrity

Tenant-owned records use UUID defaults, tenant uniqueness, composite foreign keys, and auth.jwt() RLS. Signed manifests, published/rejected versions, capability declarations, dependencies, entry points, signatures, validation decisions, and audit evidence are database-immutable. Publisher claims remain unverified until governed evidence is approved.

Implementation schema: MASS-APP-015-V01_Migration_Reference.sql. Machine contract: MASS-APP-015-V01_Plugin_Manifest.schema.json.

12. V1 Implementation

V1 supports manifest submission, JSON-schema validation, publisher provenance references, nine capability types, dependency declarations, compatibility checks, signed-version references, validation findings, registry discovery, and tenant eligibility advice. It does not install or invoke plugins.

13. Future Evolution

Future marketplace presentation, remote attestation, additional capability types, and partner trust networks attach through versioned manifest extensions. Unknown required fields fail validation; optional namespaced extensions remain preservable.

14. Constitutional Boundary Statement

V01 recognizes, validates, registers, and exposes declarations. It cannot install, activate, invoke, grant permissions, verify unsupported publisher claims, bypass authority, or approve itself.